Ders 10 Genişletilmiş Notu: Feature Envy, Video Kiralama Örneğinde Adım Adım Refactoring ve Tasarım Kalıbı Seçimi

Ders 10 Genişletilmiş Notu: Feature Envy, Video Kiralama Örneğinde Adım Adım Refactoring ve Tasarım Kalıbı Seçimi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Feature Envy (Özellik Kıskançlığı)
- 2. Bütüncül Refactoring Örneğine Giriş
- 3. Adım 1: Extract Method ile `amountFor`
- 4. Eclipse'in Extract Method Aracı
- 5. Adım 2: Değişkenlerin Yeniden Adlandırılması
- 6. Adım 3: Move Method ile `Rental.getCharge`
- 7. Yeni Metoda Test Yazılmalı mı? Black Box ve White Box
- 8. Adım 4: Replace Temp with Query
- 9. Adım 5: Bonus Hesabının Rental'a Taşınması
- 10. Adım 6: `getTotalCharge` ve `getTotalBonus`
- 11. Adım 7 ve 8: Hesaplamaların Movie'ye Taşınması
- 12. Adım 9: Customer ile Movie Arasındaki Doğrudan Bağlaşımın Kaldırılması
- 13. Adım 10: Replace Type Code with Subclasses ve Geriye Doğru Uyumluluk
- 14. Alternatif: Replace Type Code with State/Strategy
- 15. Birinci Ara Sınav Soruları Üzerine Tasarım Kalıbı Tartışması
- Örnek Sorular
- Dersin Toparlaması

1. Feature Envy (Özellik Kıskançlığı)

Ders, önceki derste başlanan kod kusurları (code smell) listesinin sonuncusu olan Feature Envy ile başlar. Hoca bu kusurdan sonra bütüncül bir refactoring örneğine geçileceğini söyler.

1.1 Tanım (ekrandan, 00:10:00)

Slayttaki tanım şudur: bir metot ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa bu duruma "özellik kıskançlığı" adı verilir. Hoca, İngilizcedeki "feature" sözcüğünün burada daha çok üye alan (member field) anlamına geldiğini, bu yüzden "özellik" diye çevirdiğini söyler. Kıskançlığın özü bir üye alan üzerinden kurulur: metot, başka bir sınıfın üye alanlarını "kıskanır".

Hoca bunun daha dar odaklı bir biçimini de tarif eder: bir üye alana kendi türünden nesnelerden çok farklı türden nesneler erişiyorsa bu da bir tür Feature Envy'dir. Ancak erişen farklı türlerin sayısı arttıkça bu kıskançlığı düzgün biçimde gidermenin bir yolu kalmaz.

1.2 “Çok” ne demek?

Hoca slayta tanımı derste yazarak tamamlar: “çok” sıfatı, o koddaki ortalama farklı sınıf üyesi erişim sayısının üzerinde bir sayıyı ifade eder. “Kime göre çok?” sorusuna hoca genelde “ortalamanın üstü” diye yanıt verir. Ama yüzde elli sınırı bile fazla görünebilir; %25 ya da %20 de kodun “bir şeyler söylemek istediğinin”, yani bir Feature Envy olabileceğinin işareti olabilir.

Bu eşik, yazılımı geliştirirken uygulanmasını gerekli gördüğünüz kod kalite ölçütlerinde tanımlanabilir. Siz %50 değil de %30 derseniz erişimleri o eşiğin altında tutmaya çalışırsınız. Hoca, bağlaşımın arttığı durumlarda bu eşiğin daha sıkı tutulmasının anlamlı olabileceğini ekler.

1.3 Kök neden: sorumluluk ataması

Feature Envy bir belirtidir; ifade ettiği kök neden sorumlulukların doğru atanmamasıdır. Bağlaşım nedeniyle her sınıf elbette başka sınıfların üyelerine erişecektir. Hiçbir sınıf yalnız kendi üye alanları ve kendi metotları üzerinden çalışmaz; öyle olsaydı bu kez uyum (cohesion) ilkesini olumsuz etkilemiş olurduk. Belli bir noktaya kadar “kıskançlık” sağlıklıdır. Ama erişim düzeni iç kaynaklardan çok dış kaynaklara yönelmişse orada bir sorun vardır.

Hoca, üye alanların genelde doğru yerlere atandığını söyler: bir varlığı modellerken o varlığın sıfatlarını, içerdiği bilgileri keşfetmek daha kolaydır. Sorumluluklar ise metotlarla gerçeklenir ve hata çoğunlukla burada yapılır. Yanlış yapılmışsa bir metodun başka bir sınıfın üyelerine yoğun biçimde eriştiği ortaya çıkar.

Slaytta ayrıca Strategy ve Visitor kalıplarının bir özellik kıskançlığı durumu olarak yorumlanmaması gerektiği yazılıdır (ekrandan, 00:10:00).

> **Ek bilgi (derste anlatılmadı):** Strategy ve Visitor kalıplarında bir nesnenin verisini başka bir sınıfın metotları işler; bu, değişen davranışı ayrı bir hiyerarşiye bilinçli olarak taşımanın sonucudur. Bu yüzden bu kalıplarda görülen yoğun dış erişim büyük olasılıkla bir tasarım kararı sayılır, kusur sayılmaz.

1.4 Çözüm

Çözüm için olası refactoring eylemleri slayta göre şunlardır:

- **Move Method:** metodun tamamı kıskanılan üyelerin bulunduğu sınıfa taşınır.
- **Extract Method:** kıskançlık metodun yalnız bir kısmındaysa o kısım çıkarılıp kıskanılan üyelerin bulunduğu sınıfa taşınabilir.

Üye alanı taşımak da söz konusu olabilir, ama yukarıdaki nedenle daha seyrektir. Hoca bu kusuru bütüncül örnekte ayrıca göstereceğini söyler.

2. Bütüncül Refactoring Örneğine Giriş

Slayttaki başlık “Bir tasarımın ardışıl düzenlemeler ile değiştirilmesi”dir: şimdiye kadar incelenen düzenlemelerin bazılarının birlikte kullanılarak yapısal programlama izleri taşıyan bir tasarımın nasıl değiştirildiğini görmek için Fowler’ın *Refactoring* kitabının 1. bölümü incelenebilir (ekrandan, 00:10:00).

2.1 Senaryo

Önceki derste anlatılan video kaset kiralama örneği hatırlatılır. Bir müşteri (`Customer`) sıfır ya da daha çok kira kaydına (`Rental`) sahiptir. Her kira kaydı bir filmle (`Movie`) ilişkilidir. Her filmin bir fiyat kodu vardır; ücret fiyat koduna ve kiralama süresine göre değişir. `statement()` metodu kiralanan filmleri, toplam borcu ve kazanılan bonus puanı içeren makbuz metnini bir `String`’de biriktirir; bu metin ekrana yansıtılınca “makbuz basılmış” olur. Birim testleri önceki derste yazılmış ve çalıştırılmıştı, bu yüzden refactoring eylemlerine başlanabilir.

2.2 Çalışma düzeni

Hoca her adımı Eclipse’te ayrı bir pakette tutar: `fowler_00` başlangıç kodudur, `fowler_01`, `fowler_02`, … her biri bir adım sonrasıdır; ayrıca `fowler_10_alt` ve `fowler_11_alt` paketleri vardır

(ekrandan, 00:30:00). Test sınıfları (`TestAll`, `TestCustomer`, `TestMovie`) her yeni pakete kopyalanır; hoca karşılaştırma aracıyla bunların arasında yalnız paket adının değiştiğini gösterir. Birim testleri kesinlikle değiştirilmez. Her adımda iki paket Eclipse'in iki yönlü karşılaştırma (two-way compare) aracıyla yan yana açılır ve farklar gösterilir.

Geçen hafta bütün kod incelenmişti ve `statement()` en kalabalık, "devasa" metot olarak dikkat çekmişti. Bütün kokuların odak noktası budur. Onu düzeltirken başka yerlere taşınan, daha az kötü kokan kısımlar da düzeltilecek; 10-11 işlemle tasarım daha iyi bir yere getirilecektir. Refactoring metodun herhangi bir yerinden başlatılabilir; hoca aynı sonuca varmak için Fowler'ın sırasını izlediğini, araya kendi uygun gördüğü bir iki adımı eklediğini söyler.

2.3 Başlangıç noktası: switch-case

`statement()` içindeki en uzun parça switch-case bloğudur ve oradan başlanır.

> **Hocanın vurgusu:** Bir yerde switch-case ya da if-else zinciri yazdığınızı görüyorsanız, büyük ihtimalle orada bir kalıtım hiyerarşisi kurup çok biçimliliği (polimorfizm) kullanmanız gerekiyordu ama yapmamışsınızdır.

Değişiklikler küçük adımlarla yapılacaktır. Bunun bir yararı da, bir adımda hata ortaya çıkarsa onu kolayca geri alabilmektir.

3. Adım 1: Extract Method ile `amountFor`

3.1 Ne çıkarılıyor?

Switch-case filmin fiyat kodunu alıp ilgili kira işlemi için ne kadar borç oluştuğunu hesaplar; bu aşamada bonusla ilgili bir şey içermez. Bloğun yaptığı esas iş bir kira kaydının borcunu bulmaktır. Switch'in anahtarı filmin fiyat kodudur, ama case'lerin içinde kira kaydıyla ilgili işlemler yapılır. Bu yüzden çıkarılacak metodun parametresi bir kira kaydıdır (`each`). Bu mantıklıdır da: `while` döngüsü her kira işlemi için borcu hesaplar; belirli bir kira işlemi bu metoda verilir, metot da borcu hesaplar. Geriye `thisAmount`'a eklenecek `double` değer döner.

3.2 Sonuç kod (ekrandan, 00:20:00)

Karşılaştırmada solda `fowler_00`, sağda `fowler_01` vardır. Switch-case `statement()` içinden kesilmiş, sınıfın sonuna yeni bir metot olarak eklenmiştir:

```
private double amountFor( Rental each ) {
    double thisAmount = 0;
    //determine amounts for each line
    switch (each.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        thisAmount += 2;
        if (each.getDaysRented() > 2)
            thisAmount += (each.getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        thisAmount += each.getDaysRented() * 3;
        break;
    case Movie.CHILDRENS:
        thisAmount += 1.5;
        if (each.getDaysRented() > 3)
            thisAmount += (each.getDaysRented() - 3) * 1.5;
        break;
    }
    return thisAmount;
}
```

Metot `private` yapılmıştır: dışarıdan bir kira kaydı verip bu metodu çağırmanın anlamı yoktur; iç çalışmayı iyileştirmeye yönelik bir metottur. Switch'in her case'i `thisAmount`'a eriştiği için `thisAmount` yeni metotta bir geçici değişken olarak tanımlanmıştır. Switch-case kodunun

kendisi değişmemiştir. Hoca, slayttaki sınıf şemasında metodun yanlışlıkla + (public) işaretiyle çizildiğini fark eder ve `private` olması gerektiğini not eder.

Adımdan sonra `fowler_01` paketindeki testler çalıştırılır ve başarılı olur: refactoring eylemi yeni bir hata ortaya çıkarmamıştır.

3.3 Sıra diyagramı

Hoca her adımda bir sıra diyagramı (sequence diagram) da gösterir. Dersin amacı sıra diyagramı öğretmek değildir; ileride yapılacak bir adımın doğrudan ilişkileri nasıl azalttığını göstermek için bu şemalar çizilmiştir. Bu adımda `statement()` her kira kaydı için kendi içinde `amountFor`'u çağırmaya başlar; geri kalan işlemler eskisi gibi sürer.

4. Eclipse'in Extract Method Aracı

Hoca, Extract Method'un Eclipse'in Refactor > Extract Method menüsüyle de yapılabildiğini gösterir. Ders notlarını yıllar önce hazırlarken Eclipse'in bu işi düzgün yapamadığını, şimdiki sürümde (Haziran 2019 sürümü) düzelip düzelmediğini merak ettiğini söyler.

4.1 İlk deneme

Önce seçim döngünün ortasından başlatılınca Eclipse işlemi yapmaz: seçimde `thisAmount` ve `each` gibi yerel değişkenler vardır ve araç `each`'i parametre olarak vermesi gerektiğini çıkaramaz. Seçim değiştirilince araç çalışır: `private` bir metot oluşturur, ancak `thisAmount`'u da gereksiz yere parametre yapar ve metoda `extracted` adını verir. Testler yine başarılıdır, ama hoca `thisAmount`'un parametre olmasına gerek olmadığını, yerel bir geçici değişken olabileceğini söyler. Bunu elle düzeltir (parametreyi kaldırıp metot içinde `0` ile başlatır), testleri yeniden çalıştırır, sonra Refactor > Rename ile adı `amountFor` yapar.

Eclipse'in diyalogunda metodun adı ve parametrelerin adı değiştirilebilir, ama "bu parametreyi geçici değişken yap" denemez. Hoca, karmaşık olmayan bu tür bir eylemde aracın yaptığı bir iki işi düzeltmektense hepsini kendisinin yapmayı tercih ettiğini söyler.

4.2 Bir öğrencinin önerisi

Soru: `double thisAmount = 0;` satırının yeri değiştirilip seçime dahil edilirse Eclipse onu yerel değişken olarak alabilir mi?

Cevap: Hoca dener: `thisAmount` tanımı seçimin içine alınıp Extract Method çalıştırılınca araç `thisAmount`'u yerel değişken olarak alır ve yalnız `each`'i parametre yapar. Hoca bunun güzel bir ipucu olduğunu söyler. Notları hazırlarken bir konferans bildirisinde Eclipse'in karmaşık refactoring eylemlerini çok başarılı yapamadığını okuduğunu, bu yüzden daha fazla uğraşmadığını; bu öneriyle sorunun çözüldüğünü ekler.

5. Adım 2: Değişkenlerin Yeniden Adlandırılması

İkinci adımda `amountFor` içindeki değişken adları düzenlenir: `each` yerine `aRental`, `thisAmount` yerine `result` (ekrandan, 00:40:00).

```
private double amountFor( Rental aRental ) {
    double result = 0;
    //determine amounts for each line
    switch (aRental.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        result += 2;
        if (aRental.getDaysRented() > 2)
            result += (aRental.getDaysRented() - 2) * 1.5;
        break;
    ...
```

Değişkenlerin daha anlamlı adlarla yeniden adlandırılması da bir refactoring eylemidir. Bir kod kusuruna yönelik değildir ama anlaşılabilirliği artırır; günün sonunda kodu daha iyi anlamayı sağlar. Bu adım Eclipse'in Refactor > Rename aracıyla kolayca yapılır. Karşılaştırmada yalnız adların değiştiği görülür. Hoca, testleri çalıştırmak bu kadar kolay olmasaydı yalnız ad değişikliği için test çalıştırmaya gerek görmeyeceğini, ama iki üç tıklamayla çalıştığı için yine çalıştırdığını söyler.

6. Adım 3: Move Method ile `Rental.getCharge`

6.1 Yeni kusuru görmek

Bir refactoring eylemi yeni kod kusurları ortaya çıkarabilir. Bu yüzden en son yapılan değişikliğe yeniden bakmak yararlıdır. `amountFor` incelendiğinde: `aRental` parametre olarak gelir; kira kaydının hangi filmle ilişkili olduğuna, filmin fiyat koduna, kira kaydının gün sayısına bakılır. Metot çoğunlukla kira kaydının alanlarına, bir de `Movie` nesnesinin bir alanına getter'larla erişir. `Customer` sınıfının hiçbir üye alanına erişmez. Bu bir Feature Envy'dir.

6.2 Taşıma

Metot kendi sınıfının üyeleriyle çalışmadığına ve ağırlıkla kira kaydının alanlarına eriştiğine göre `Rental` sınıfına taşınmalıdır (Move Method). Taşınınca kira kaydı nesnesini parametre olarak vermeye gerek kalmaz; metot zaten o nesnenin kendisinde çalışır ve filmine, gün sayısına erişebilir. Hoca taşırken adını da `getCharge` yapar: o kira kaydının bedelini döndürür. Bu işlem Eclipse ile de yapılabilir; hoca zaman kazanmak için hazır sonucu gösterir.

Karşılaştırmada `Customer.statement()` içinde `amountFor(aRental)` yerine `aRental.getCharge()` çağrılır ve `amountFor` metodu `Customer`'dan kalkmıştır. `Rental` sınıfına yalnız `getCharge()` eklenmiştir. Kod biraz daha derli toplu olmuştur, ama switch-case hâlâ durmaktadır; sonunda o da elenecektir. `fowler_03` testleri sorunsuz geçer.

Hoca `getCharge`'ı `public` yapmıştır; paket düzeyinde de kalabilirdi, ama herhangi bir kira işleminin bedelinin dışarıdan sorulabilmesinde sakınca görmez.

7. Yeni Metoda Test Yazılmalı mı? Black Box ve White Box

Move Method ile `Rental`'a yeni bir metot eklenmiştir. Hoca bunun için ayrı bir birim testi (test case) yazılıp yazılmayacağını tartışır. Yazmak zorunlu değildir; isterseniz yazabilirsiniz, bu durumda white box test ilkesiyle çalışıyorsunuz demektir.

- **Black box test:** sınanan şeyin iç yapısı bilinmez. Yalnız verilen girdiye karşılık beklenen çıktının elde edilip edilmediğine bakılır.
- **White box test:** kodun dallanmaları ve döngüleri üzerinden testler yazılır. Hedef kod kapsamasıdır (code coverage): yürütülen testlerin yazılımın kodunun ne kadarının üzerinden geçtiği.

Kod kapsamasını yüksek tutmak elbette istenir. Ama bir projede sınama için belirli bir zaman ayrılmıştır ve o sınır aşılmamaya çalışılır. Hoca örnek girdi kümesini ve buna karşılık beklenen çıktı kümesini zaten elde ettiğini, refactoring'in bunu değiştirmediğini söyler. Test maliyetini artırmamak için black box yaklaşımıyla devam eder. Yüksek riskli bir projede ise white box testi düşünüp ona odaklanan test durumları yazmak daha doğru bir karar olurdu.

8. Adım 4: Replace Temp with Query

8.1 Hangi geçici değişken kalkabilir?

`Rental.getCharge`'daki switch-case hâlâ rahatsız edicidir, ama daha büyük sorun `statement()`'ın hâlâ uzun ve karmaşık olmasıdır. Geçici değişkenler metodu uzatan önemli bir etkendir; onları kaldırmak için Replace Temp with Query kullanılabilir. Hoca değişkenleri tek tek değerlendirir:

- `totalAmount`: kümülatif toplamdır, bu aşamada dokunulamaz.
- `frequentRenterPoints` (bonus puanı): o da kümülatif toplamdır.

- `result`: metodun döndüreceği metindir.
- `thisAmount`: bir kez `aRental.getCharge()` ile atanır ve değeri hiçbir yerde değişmez. Değişseydi kaldırmak daha karışık olurdu. İki yerde kullanılır.

8.2 Sonuç (ekrandan, 00:49:10)

`thisAmount` kaldırılır ve kullanıldığı iki yerde doğrudan `aRental.getCharge()` çağrılır. Karşılaştırmada solda `fowler_03`, sağda `fowler_04` vardır:

```
// fowler_03
double thisAmount = 0;
Rental aRental = (Rental) rentals.nextElement();
thisAmount = aRental.getCharge();
...
String.valueOf(thisAmount) + "\n";
totalAmount += thisAmount;

// fowler_04
Rental aRental = (Rental) rentals.nextElement();
...
String.valueOf(aRental.getCharge()) + "\n";
totalAmount += aRental.getCharge();
```

8.3 Başarım tartışması

Bu işlem her kira kaydı için yapılır ve `getCharge` iki kez çağrılır. Hoca bir müşterinin kaç film kiralayabileceğini sorar: milyon değil, beş, on tane belki. On kira kaydı için yirmi metot çağrısı olur. `getCharge` basit bir getter değildir, biraz hesap yapar, ama çalışma zamanı başarımına katkısı en düşük düzeyde, hissedilmeyecek kadar küçük kalacaktır. Testler başarılıdır.

9. Adım 5: Bonus Hesabının Rental'a Taşınması

9.1 Ad değişikliği ve yöntem (ekrandan, 01:00:00)

Slayttaki beşinci adım: bonus hesaplaması için metot çıkarma. Bonus (`frequentRenterPoints`) kiralama işlemiyle ilgili olduğu için bu işlemin `Rental` sınıfına alınması yerinde olacaktır. `frequentRenterPoints` çok uzun bulunmuş, yerine `bonus` kullanılmıştır. Hoca, "bonus"un dilimize yerleşmiş, video oyunlarında kazanılan ek puan için de kullanılan bir sözcük olduğunu söyler. Slayta göre UML şemalarında amaç refactoring'i vurgulamak olduğundan her getter/setter metodu gösterilmemiştir.

Bonus hesabı borç hesabına çok benzer şekilde yapılır: her kira işlemi için bir puan, yeni çıkan bir filmin bir günden uzun kiralanması için bir puan daha kazanılır. Hesap kiralanan filmin tipine ve kira süresine ihtiyaç duyar. Borç için yapılan tartışma bonus için de geçerli olduğundan, iki adımda (önce Extract Method, sonra Move Method) yapmak yerine doğrudan "çıkar ve taşı" yapılır.

9.2 Önceki kod (ekrandan, 01:05:00)

`fowler_04`'teki `statement()` içinde bonus hesabı şöyledir:

```
// add frequent renter points
frequentRenterPoints ++;
// add bonus for a two day new release rental
if ((aRental.getMovie().getPriceCode() == Movie.NEW_RELEASE)
        && aRental.getDaysRented() > 1) frequentRenterPoints ++;
```

9.3 Sonuç

`statement()` içinde artık yalnız kira kaydının bonusu alınıp toplam bonusa eklenir: `bonus += aRental.getBonus();` (ekrandan, 01:10:00). Kümülatif toplam olduğu için `bonus` değişkeni si-

linemez. `Rental`'a eklenen metot (ekrandan, 01:17:00):

```
public int getBonus( ) {
    if ((getMovie().getPriceCode() == Movie.NEW_RELEASE)
            && getDaysRented() > 1)
        return 2;
    else
        return 1;
}
```

Hoca burada küçük bir ek düzenleme de yaptığını belirtir: "önce bir artır, koşul sağlanırsa bir daha artır" biçimi aynen taşınsaydı yeni metotta da bir geçici değişken gerekecekti. Bunun yerine koşul aynen bırakılıp koşul sağlanırsa `2`, değilse `1` döndürülür; böylece metot geçici değişken içermez. Testler yeşil çubukla geçer.

10. Adım 6: `getTotalCharge` ve `getTotalBonus`

10.1 Kalan geçici değişkenler

Hoca `totalAmount` ve `bonus` değişkenlerinin nasıl kaldırılacağını derste hemen hatırlayamaz ve hazır sonuca bakar: `Customer` sınıfına `getTotalCharge()` ve `getTotalBonus()` metotları eklenmiştir (ekrandan, 01:10:00). Toplamlar elbette bir yerde biriktirilecektir, ama en azından `statement()` metodunun içinden çıkarılmıştır: ilgili yerel değişkenler ilgili metotlara taşınmıştır.

Bu yine Replace Temp with Query'dir, ama önceki adımdan farklıdır. 4. adımda geçici değişkenin yerine hazır bir sorgunun (`getCharge`) sonucu yazılmıştı. Burada toplamları bulacak hazır bir metot yoktur; önce her iki geçici değişken için birer sorgu metodu çıkarılır (Extract Method), sonra kullanılır. Böylece hem `statement()` kısalır hem geçici değişkenlerden kurtulunur.

10.2 Sonuç kod (ekrandan, 01:10:00)

`fowler_06`'da `statement()`'ın başı şöyledir; `totalAmount` ve `bonus` tanımları kalkmıştır:

```
public String statement( ) {
    Enumeration<Rental> rentals = _rentals.elements();
    String result = "Rental Record for " + getName() + "\n";
    while (rentals.hasMoreElements()) {
        Rental aRental = (Rental) rentals.nextElement();
        //show figures for this rental
        result += "\t" + aRental.getMovie().getTitle()+ "\t" +
        String.valueOf(aRental.getCharge()) + "\n";
    }
    ...
```

Yeni sorgu metotlarında biriktirici değişkenin adı `result` kalmıştır. Hoca metot adından ne biriktirildiği anlaşıldığı için `result`'un da olabileceğini, ama `totalCharge` ve `bonus` gibi adların belki daha doğru olacağını söyler; bu değişiklik yapılırsa sonraki bütün paketlerde de yapılmalıdır.

10.3 Customer'ın durumu

`Customer` sınıfına genel bakıldığında metotlar kısalmıştır. `statement()` nispeten uzun görünse de eskisi kadar uzun değildir ve derli topludur. Hoca, alt bilgi (footer) satırlarının da bir Extract Method ile ayrı bir metoda çıkarılabileceğini, ama bunun henüz yapılmadığını söyler. Bunun dışında `statement()`'ta rahatsız edici bir şey kalmamıştır; şimdi `Rental` tarafında yeni kusur oluşup oluşmadığına bakılır.

11. Adım 7 ve 8: Hesaplamaların Movie'ye Taşınması

11.1 Switch-case'in asıl sahibi

`Rental.getCharge()` filmin fiyat koduna bakar ve case'lerde bunu sınar; ayrıca kira kaydının gün sayısına bakar. Bir metodun kendi sınıfının üye alanına ek olarak başka bir sınıfın üye alanına erişmesi tek başına sorun değildir. Asıl sorun, filmin üye alanına switch-case'in anahtarı olarak erişilmesidir: kritik karar filmin hangi türden olduğudur ve fiyat hesabı ağırlıklı olarak filmin koduyla ilgilidir, kiralama süresiyle değil.

Ayrıca `Movie` sınıfı şu an basit bir veri sınıfıdır (data class): üye alanları, getter/setter metotları ve kurucusu vardır, hiçbir hesap yapmaz. Bu yüzden `getCharge`'tan bir Extract + Move Method yapılıp `Movie`'ye sorumluluk eklenir. `Movie`'nin bedeli hesaplayabilmek için yalnız kaç gün kiralandığı bilgisine ihtiyacı vardır; `Rental` bu bilgiyi filme iletir ve "şu kadar gün kiralandın, ne kadar ücretlendirilmen gerekiyor?" diye sorar.

11.2 Sonuç kod (ekrandan, 01:17:00, 01:30:00)

`fowler_07`'de `Rental.getCharge()` tek satıra iner:

```
public double getCharge( ) {
    return _movie.getCharge( _daysRented );
}
```

`Movie` sınıfının başı ve yeni metodun başı (`fowler_09` paketinden; `getCharge` 7. adımdan beri aynıdır):

```
public class Movie {
    public static final int CHILDRENS = 2;
    public static final int REGULAR = 0;
    public static final int NEW_RELEASE = 1;
    private String _title;
    private int _priceCode;
    public Movie(String title, int priceCode) {
        _title = title;
        _priceCode = priceCode;
    }
    public int getPriceCode() {
        return _priceCode;
    }
    public void setPriceCode(int arg) {
        _priceCode = arg;
    }
    public String getTitle (){
        return _title;
    }
    public double getCharge( int daysRented ) {
        double result = 0;
        switch ( getPriceCode() ) {
        case Movie.REGULAR:
            result += 2;
            if (daysRented > 2)
                result += (daysRented - 2) * 1.5;
            break;
        ...
```

Switch-case hâlâ vardır, ama artık ağırlıkla kendi sınıfının nesnesinin üye alanına erişerek çalışır.

11.3 Kazanç: yeni film türü eklemek

Hoca bir gereksinim değişikliği varsayar: yeni bir film türü, örneğin Blu-ray eklenecektir. 7. adımdan sonra bu değişiklik yalnız `Movie` sınıfında yapılır: yukarıya `public static final int`

biçiminde değeri 3 olan yeni bir sabit ve switch'e yeni bir case eklenir. Eski sürümde kalınsaydı hem `Movie`'ye bir tip, hem `Rental`'a bir case eklemek gerekecekti. Önceki derste bunun gibi birden çok sınıfa dağılan değişikliklere Shotgun Surgery dendiği hatırlatılır.

11.4 Adım 8: `getBonus`

`fowler_07` testleri sorunsuz geçtikten sonra aynı işlem bonus için yapılır. `Rental.getBonus()` içindeki, filmin `NEW_RELEASE` olup olmadığını sınayan iş mantığı `Movie.getBonus(int daysRented)` metoduna taşınır; `Rental` da `getCharge`'taki gibi filmin `getBonus`'unu kendi gün sayısıyla çağırır. Hoca if bloğunun içine müdahale etmediğini, aşırı karmaşık olmadığı için aynen taşıdığını söyler. Böylece Feature Envy durumları giderilmiş olur. Testler yine sorunsuzdur.

12. Adım 9: Customer ile Movie Arasındaki Doğrudan Bağlaşımın Kaldırılması

12.1 Sıra diyagramından okunan kusur (ekrandan, 01:22:40)

Sıra diyagramına ana hatlarıyla bakıldığında `Customer` genelde `Rental` ile, `Rental` da `Movie` ile konuşur. Ama bir yerde `Customer`'dan `Movie`'ye doğrudan bir erişim vardır: `getTitle`. Slayttaki dokuzuncu değişiklik budur: 8. adıma kadar yapılan değişiklikler sonucunda `Customer` ile `Movie` arasındaki tek doğrudan ilişki `getTitle` mesajı olmuştur; daha düşük bağlaşım için bu erişim `Rental` üzerinden sağlanabilir.

Filmin adını öğrenmek için `Customer`'ın doğrudan filme sormasına gerek yoktur. Kira kaydı filmle ilişkili olduğu için `Customer` kira kaydına "ilişkili olduğun filmin adı ne?" diye sorabilir.

12.2 Değişiklik

`Rental`'a `getMovieTitle()` metodu eklenir; metot yalnız `getMovie().getTitle()` döndürür. `Customer.statement()` içindeki `aRental.getMovie().getTitle()` çağrısı `aRental.getMovieTitle()` olur. 8 ile 9 arasındaki tek fark budur.

Sonuçta şema katmanlı hale gelir: `Customer` hep `Rental` ile, `Rental` gerektiğinde `Movie` ile konuşur. `Customer`'dan `Movie`'ye doğrudan bağımlılık kalkar; dolaylı ilişki tasarım gereği sürer (`Customer` → `Rental` → `Movie`). Önceki sürümde bu erişim üye alan üzerinden değil, metot çağrısı olarak vardı ve sınıf şemasında kesikli çizgili bir kullanım okuna karşılık geliyordu.

13. Adım 10: Replace Type Code with Subclasses ve Geriye Doğru Uyumluluk

13.1 Kalan kusur

9. adımdan sonra `Customer`'da (footer'ın çıkarılması dışında) ve kısa, öz metotlardan oluşan `Rental`'da rahatsız edici bir şey kalmamıştır. `Movie`'deki switch-case ise hâlâ rahatsız edicidir. Bu switch-case'i çözmenin tek yolu bir kalıtım ilişkisi kurmaktır. Bunun bir iki alternatifi vardır.

13.2 Alt sınıflarla çözüm (ekrandan, 01:30:30)

İlk tasarım: soyut bir `Movie` sınıfı ve her fiyat kodu için bir alt sınıf. `getCharge` ve `getBonus` soyut bırakılır, çünkü farklı tür filmlerde farklılaşan işlemlerdir; alt sınıflarda kodlanır. `fowler_10` paketinde `Movie`, `MovieChildrens`, `MovieNewRelease`, `MovieRegular` sınıfları vardır:

```
public abstract class Movie {
    private String _title;
    public Movie(String title ) {
        _title = title;
    }
    public String getTitle (){
        return _title;
    }
```

```
    public abstract double getCharge( int daysRented );
    public abstract int getBonus( int daysRented );
}
```

`Movie` yalnız bütün türlerde ortak olan başlık bilgisini, kurucuyu ve getter'ı tutar. Her case'in içindeki davranış ilgili alt sınıfa taşınmıştır.

13.3 Bedel: arayüz ve testler

Bu eylem dış arayüzü değiştirmiyormuş gibi görünse de aslında değiştirir. Önceden `Movie` nesneleri doğrudan, türü verilerek oluşturuluyordu ve `TestMovie` sınıfı da öyle yazılmıştı. 9'dan 10'a geçerken test sınıfları kopyalanıp yapıştırılamaz: `getPriceCode` metodu ve `Movie(String, int)` kurucusu yoktur. Hoca `getPriceCode`'u soyut bir metot olarak ekleyip alt sınıflarda somut değer döndürmeyi düşünebileceğini, ama bunun da sakıncalı olduğunu söyler: "sihirli sayılardan" (magic number) ve tip kodlarından olabildiğince uzaklaşılmaya çalışılmaktadır. Geriye doğru uyumluluk için küçük bir taviz olarak eklenebilirdi, ama kurucu imzası sorunu yine kalır.

13.4 Sürüm kararı

Hoca bunu geniş, sahada müşterilerin kullandığı bir yazılım olarak düşünmeyi önerir. Filmler bir veri tablosunda tutuluyor olsun. Tasarım değişikliği veri tabanı yapısını da etkileyecekse geriye doğru uyumluluk kaybolur; oysa refactoring'in önemli bir kazanımı geriye doğru uyumluluğu korumaktı. Bu yüzden ciddi bir karar verilmelidir: 9 adımı yapıp yazılımı örneğin 2.0'dan 2.2'ye mi geçirmeli, yoksa 10. adımı da yapıp 3.0 ana sürümüne geçip geriye doğru uyumluluğu kaybetmeli mi? Bu karar yazılımın hitap ettiği pazara, şimdiki ve kazanılmak istenen müşterilere bağlıdır; yalnız teknik değil, mali ve pazarlama yönleri de vardır. Avantajlar ve dezavantajlar tartılarak karar verilir.

> **Hocanın vurgusu:** Bunu en başta fark edebilseydik, yani tip kodlarını görüp yazılımı sahaya sürmeden önce Replace Type Code with Subclasses yapsaydık iyi olurdu.

Hoca 9'dan 10'a geçip geçmeme değerlendirmesini çevresel etkenleri de düşünmeleri için öğrencilere bırakır.

14. Alternatif: Replace Type Code with State/Strategy

14.1 Fikir

Diğer alternatif: `Movie` sınıfı kalır, ama `Price` adlı ayrı bir sınıf hiyerarşisi oluşturulur. Hoca bu tasarımı bir ara ders notlarından çıkardığını, çünkü Strategy kalıbı derste kod kusurlarından sonra işlendiği için uygun görmediğini; artık sıra değiştiği için rahatlıkla gösterebileceğini söyler. Strategy kalıbı kullanılır; `Movie`'deki üye alanlar (tip kodu) ister istemez kalır. Bu çözümle `Customer` testlerini kopyalarken çıkan kurucu uyumsuzluğu oluşmaz: tip kodu korunur, ama switch-case'deki hesaplar kaldırılır. Bir ara uzlaşıdır.

14.2 Kod (ekrandan, 01:35:10)

`fowler_10_alt` paketinde `Movie`, `Price`, `PriceChildrens`, `PriceNewRelease`, `PriceRegular` ve `Rental`, `Customer` vardır:

```
public abstract class Price {
    public abstract double getCharge( int daysRented );
    public abstract int getPriceCode( );
    public int getBonus( int daysRented ) {
        return 1;
    }
}
```

`Price` aslında bir arayüz olabilirdi. Ama `getBonus` hesabını bir yerde toplayabilmek için soyut sınıf yapılmıştır: varsayılan olarak bir puan verilir; yalnız yeni çıkan bir film birden fazla gün

kiralanmışsa iki puan döner. Bu yüzden `getBonus` somut bir metot olarak eklenmiştir. `PriceChildrens` `getBonus`'u override etmez, olduğu gibi kullanır; `PriceNewRelease` ise override edip ilgili bonus mantığını içerir. `PriceChildrens`, `PriceNewRelease` ve `PriceRegular`'ın `getCharge` metotlarında hiç switch-case yoktur.

Tip kodu kaldığı için ilgili `Price` nesnesini oluştururken hâlâ bir switch-case vardır; ama içinde karmaşık bedel ve bonus hesapları yapılmaz.

14.3 Karşılaştırma

Strategy kalıbını kullanan çözümde tip kodları kalır, ama birim testleri değiştirilmek zorunda kalınmaz; avantajı budur. Alt sınıflı çözümde switch tümden kalkar ama arayüz ve testler değişir. Hoca `fowler_11_alt` paketinde de bir değişiklik yaptığını, ama derste incelemeye zaman ayırmadığını söyler.

> **Ek bilgi (derste anlatılmadı):** Fowler'ın kitabında bu yaklaşım "Replace Type Code with State/Strategy" adıyla geçer; burada State kalıbının kullanılmasının nedeni, bir filmin fiyat kodunun nesne yaşarken değişebilmesidir (`setPriceCode`). Alt sınıflı çözümde bir nesnenin sınıfı oluşturulduktan sonra değiştirilemediği için bu büyük olasılıkla mümkün olmazdı.

15. Birinci Ara Sınav Soruları Üzerine Tasarım Kalıbı Tartışması

Dersin son bölümünde hoca birinci ara sınavın sorularını ve kendi çözüm notlarını açar; önce kendi düşüncesini söyler, sonra farklı düşünen öğrencilerle tartışır. Bu bölümdeki soru metinleri ve çözüm tartışması aşağıdaki "Örnek Sorular" bölümünde ayrıntılı verilmiştir. Burada tartışmadan çıkan genel ilkeler toplanmıştır.

15.1 Kalıp seçiminde genel ilkeler

- Bir kalıp yanıtında doğru kalıbın İngilizce adı verilmeli ve neden seçildiği düzgünce açıklanmalıdır. Hocanın çözüm notundaki tek cümlelik açıklamalar yalnız rehberdir; daha ayrıntılı açıklama beklenir.
- Listede olmayan bir kalıp da iyi gerekçelendirilirse uygun ya da kısmen uygun sayılabilir.
- Tasarım yaparken kod kusuru adını bilmeseniz de kopyala-yapıştır programlamanın kötü olduğunu bilmelisiniz; bir kalıp kopya kod üretiyorsa zayıftır.
- Facade, Adapter ve Mediator genelde bir sınav sorusunun ana yanıtı olarak düşünülmemelidir. Facade karmaşıklığı bir noktada soyutlar ama çözmez ve "her şeye yuvarlanabilir". Adapter çok özeldir: uymayan bir prize takabilmek için araya giren bir katman, yalnız metot imzalarını değiştiren bir delege gibidir. Bunlar ana kalıbın yanında ek bir gereksinimi karşılamak için düşünülebilecek ikincil kalıplardır. Mediator da karmaşıklığı dağıtmak yerine toplar; ona tam oturan bir soru gelebilir ama genelde ana yanıt değildir.
- MVC zorlanırsa her şeye oturtulabilir (veri, kontrol, gösterim katmanları); bu yüzden tek başına güçlü bir yanıt sayılmaz.
- Hoca kalıp sorularını kaynak kitaplardaki örneklerden esinlenerek hazırladığını söyler: birinci soru *Applied Design Patterns*, ikinci soru *C# Design Patterns: A Tutorial* kitabından esinlenmiştir (ekrandan, 02:00:00, 02:22:35). Bu kitaplardaki örnekleri okuyup çözmeye çalışmak iyi bir hazırlıktır.

15.2 Öğrencilerin yorumları

Soru: Bir öğrenci birinci soruda Abstract Factory kullandığını, derste geçen DirectX renderer örneğindeki ayrıma benzerlik gördüğü için bu kalıbı seçtiğini söyler. **Cevap:** Hoca bunun uygun bir seçim olduğunu söyler; kalıbın sınıf şemasında hangi kısmın Abstract Factory'ye karşılık geldiğinin görsel olarak ayırt edilebilmesi de beklenir.

Soru: Bir öğrenci birinci soruyu Decorator ile yaptığını; Decorator'ın tanımındaki "nesneler çalışma sırasında değiştiriliyorsa" ifadesine kandığını, değiştirmekle yeni özellik eklemeyi karıştırdığını söyler. **Cevap:** Hoca Decorator'ın kısmen uygun olduğunu, yanlış ya da kısmen uygun bir kalıp bile UML şemasına düzgün oturtulmuş ve nesneye dayalı tasarımın ana ilkelerine aykırı değilse şema kısmının değerlendirildiğini söyler.

Soru: Bir öğrenci Abstract Factory ile Adapter arasında kalıp Adapter'ı seçtiğini; farklı ülkelerin adres sınıflarının farklı kurucuları olacağını, ülke sayısı arttıkça Abstract Factory'deki sınıfların şişeceğini düşündüğünü, basit `setAdres`, `getAdres` metotlarını bir adaptörle her ülkeye uydurmayı planladığını söyler. **Cevap:** Hoca bunun Prototype'a benzer bir yaklaşım olduğunu söyler: en geniş örnek düşünülür, sonra her ülke kuralı için o en geniş örnekten ilgili ülkeye bir adaptör konur (gerekirse alanların yeri değiştirilir, eyalet kanton yapılır). Bu yüzden Adapter'ı Decorator ve Prototype gibi kısmen uygun sayar; Abstract Factory'nin daha iyi bir seçim olacağını belirtir.

Örnek Sorular

Soru 1 (ekrandan, 01:51:35; çözüm notu ekrandan, 02:00:00 ve 02:22:35)

Soru metni: Adres, tarih ve telefon numaraları ile uğraşan bir kişisel bilgi yönetimi programı hazırlıyorsunuz. Bu program bir adres defteri, kişisel planlar oluşturma, randevu yönetimi ve kişi erişim bilgisi yönetimi işlevlerini görecektir. Bu nedenle program adres, tarih ve telefon numarası verilerini çok yoğun olarak kullanacaktır. Programı yurt içindeki ve dışındaki çeşitli ülkelerden müşterilere satmayı planlıyorsunuz.

Adres, tarih ve telefon numaralarını simgeleyen sınıflar tasarlayabilirsiniz. Bu sınıflar ilgili bilgiyi saklamakla kalmayıp, biçimleri ile ilgili çeşitli iş kurallarını da içermek zorunda olacaktır. Örneğin ABD'de adres bilgisinde eyalet alanı yer almak zorundayken Türkiye'de böyle bir alan yoktur. Tarihler de değişik ülkelerde değişik biçimlerde gösterilirler. Her yeni iş kuralı ile problem alanındaki sınıflar giderek şişecek ve bakımları zorlaşacaktır. Tasarım kararlarınızı verirken bu durumu göz önünde bulundurunuz.

Hocanın sözlü çözümü: Sorunun özü farklı ülkelerdeki farklılıklardır: adres yapısı (ABD'de eyalet var, Türkiye'de yok; bazı ülkelerde kanton gibi başka idari birimler var), telefon numarası (ülke kodu ve ülkeye göre alan kodu kuralları) ve tarih biçimi (gün-ay-yıl ya da önce ay sonra gün). İlk akla gelen tasarımla bu varyasyonları sisteme eklemek zorlaşır. Hocanın aklına gelenler:

- Karmaşık bileşenlerden bir bütün oluşturma: **Builder**.
- Farklı ülkelerin farklı kuralları: **Strategy**. Adres, telefon ve tarih için üç ayrı strateji ya da üçünü birlikte ele alan tek bir strateji kurulabilir. (Strategy o tarihte henüz işlenmemişti, ama bilen öğrencilerin yanıtı geçerli sayılır.)
- **Abstract Factory**: GUI bileşenlerinin farklı pencere sistemleri için ayrı aileleri olması örneğindeki gibi; Türkiye ailesinin üyeleri Türk adresi, Türk telefonu, Türk tarihi; Amerika ailesinin üyeleri Amerikan adresi, telefonu, tarihi; Avrupa ailesi de benzer biçimde.

Hocanın çözüm notu (ekrandan, 02:00:00): Notun başlığı "Soru 1: Abstract factory: Applied Design Patterns" ve "± tam uygun değil demek" açıklamasıdır.

- + **Abstract Factory:** Aynı ürünler (tarih, tel, adres) farklı ülkelerde farklı olacak.
- + **Singleton:** Ülke kuralları (adres doğrulama vb.) sabittir. Hoca bunu kabul eder, ama yalnız "Singleton" demenin yetmediğini, her farklılık için ayrı Singleton konursa Duplicated Code kusuruna düşüleceğini söyler; Abstract Factory bu kusura düşürmez.
- + **Factory Method:** Bilgi kaydı somut işlemleri farklı ülkelerde farklı yapacak (ör. Türkiye, Amerika, Avrupa Birliği fabrikalarından adres vb. oluşturmak).
- + **Builder:** Bilgi kaydının farklı parçaları farklı ülkelerde farklı bileşimlerle oluşacak.
- + **Bridge:** Verilerin gösterimini ve oluşumunu birbirinden ayırmak. Örneğin tarihte gün, ay, yıl hep olacak ama gösteriliş biçimleri farklı olacak: tarih soyutlaması ve tarih gösterimi soyutlaması; adres için de aynı.
- + **Strategy:** Ders notlarında benzer örnek mevcut.
- + **Visitor:** Hocanın düşündüğü bir kalıp değildi, ama sorunun son cümlesi (yeni iş kuralları eklenecek) bu kalıba yönlendiriyor. Visitor'ın amacı bir parça-bütün ilişkisine yeni yeteneklerin olabildiğince esnek biçimde eklenmesine izin vermektir.
- + **Template Method:** Yapılacak işlevlerin ortak yönleri olup alt ayrıntılarının ülkeye göre farklılaşması ile.
- ± **Decorator:** En çıplak bilgi kümesi üzerine ülkeye göre ekler yapılabilir. Hocanın sözlü açıklaması: burada bir ek yetenek değil, bir farklılık söz konusudur. En küçük ortak adres yapısı çekirdek alınıp eklemelerle gidilebilir, ama çekirdek nedir? Diyelim çekirdek en

az alana sahip Türkiye oldu; daha az alanı olan yeni bir ülke gelirse Türkiye artık bir dekoratör adayı olur ve yeni ülke eklerken Türkiye'nin kodunu da değiştirmek gerekir.
- ± **Prototype:** Her ülke için geçerli olacak kuralların tümünü toplayan şişkin bir prototip; geçersiz alanlar `null` olur. Belli ülkeler için belli özellikleri atıl kalmak dezavantajı ile kullanılabilir. Decorator'ın tam tersidir.
- ± **Adapter:** Prototype ile benzer tartışma ve yöntem.
- ± **MVC:** Her şeye uyan bir kalıp; hem sadece aynı verinin değişik gösterimi (dd/mm/yy ve yy/mm/dd gibi), hem de farklı veri alanları söz konusu (adreste eyalet var/yok gibi).
- − **Flyweight:** Dışsallaştırılabilecek alan yok. "Verilerin çok yoğun kullanılacağı" ifadesi Flyweight'i düşündürebilir (çok sayıdaki nesnenin durum bilgisinin bir kısmının havuzda paylaşılması), ama Flyweight için varlığın durum bilgisinin içsel ve dışsal olmak üzere iki alt kümeye bölünebilmesi gerekir; burada böyle bir bilgi kümesi yoktur. Hoca bunun kasıtlı bir çeldirici olmadığını söyler.
- − **Singleton:** Tek bir adresi ortak kullanan olmayacak; farklı kişilerin farklı durum bilgileri olacaktır.

Soru 2 (ekrandan, 02:20:00; çözüm notu ekrandan, 02:24:40)

Soru metni: Herhangi bir kurum departmanlar ve alt departmanlar şeklinde organize olabilir. Üstelik alt departmanlar birkaç düzeyde tanımlanabilir. Örneğin üniversite fakültelerden, fakülteler bölümlerden, bölümler de anabilim dallarından oluşur. Ya da bir ülkenin silahlı kuvvetlerini genelkurmay başkanlığı komutasında toplarsak altında kuvvet komutanlıkları, bunlardan örneğin kara kuvvetleri komutanlığı emrinde ordular, ordu emrinde kolordular, kolordular emrinde tümenler, vb. yer alır. En alt düzeydeki departmanlar ise çalışanlardan oluşacaktır. Böyle bir kurumsal yapıyı modelleyerek gerek bütün organizasyon gerekse ara düzey departmanlardan biri için şu raporlamaları yapabilecek bir tasarım yapınız:

- Departmanların ve çalışanların listelenmesi
- Departmana verilen toplam aylık maaşın hesaplanması

Hocanın sözlü çözümü: Burada hiyerarşik bir yapı, bir ağaç yapısı, yani bir parça-bütün ilişkisi vardır. Soru, hiyerarşi biçimindeki bir parça-bütün yapısını modellemeyi ve bu model üzerinde çeşitli hesaplamalar yapmayı istemektedir. Parça-bütün yapısını modellemeye yönelik kalıp **Composite**'tir; asıl yanıt budur.

Hocanın çözüm notu (ekrandan, 02:24:40): Başlık "Soru 2: Composite: C# Design Patterns A Tutorial'dan esinlenme".

- + **Composite:** Hiyerarşik parça-bütün ilişkilerini modelleyen kalıptır.
- + **Visitor:** Düşünülen kalıp değildi, ama parça-bütünü dolaşıp farklı raporlamalar için uygun. Hoca, Visitor'ın amacının zaten var olan bir yapıda dolaşmak olduğunu, parça-bütün ilişkisini kendisi modellemediğini de hatırlatır.
- ± **Prototype:** Departmanlar ve çalışanlar için ayrı prototype kalıbı kurulmalı. Ağaçta kök, ara düğüm ve yaprak düğümün ayrı özel durumları vardır; yaprak (çalışan) düğümün çocuğu olamaz. Tek bir prototip kullanılırsa ara düğümdeki çocuk bilgisi yaprak için anlamsız olur ve bu anlamsız veri bazı prototip nesnelerine aktarılmak zorunda kalınır.
- ± **Iterator:** Sadece raporlama gereksinimini karşılar, parça-bütünü karşılamaz. Iterator'ın amacı yalnız dolaşmaktır ve illa ağaçta dolaşması gerekmez, bir listede de dolaşabilir.
- ± **Chain of Responsibility:** Aşağıdan yukarıya zincir kurar, yukarıdan aşağıya bağlantısı yoktur. Emir-komuta zinciri adı benzerlik çağrıştırır, ama bu kalıpta yanıtı zincirdeki tek bir nesne verir; oysa toplam maaş için bütün bileşenler dolaşılıp maaşlar kümülatif toplamda biriktirilmelidir. Ayrıca zincir doğrusal bir ilişkidir.

Soru 3 (derste sözlü, 00:43:32)

Soru: Move Method ile `Customer`'dan `Rental`'a taşınan `getCharge` metodu için ayrı bir birim testi yazmak gerekir mi?

Cevap (derste verildi): Gerekmez. Testler black box mantığıyla yazılmıştır: örnek girdi kümesi ve beklenen çıktı kümesi refactoring'den etkilenmez. Yeni metoda özel test yazmak white

box yaklaşımıdır ve kod kapsamasını artırır, ama projenin test için ayrılmış zamanını da artırır. Yüksek riskli bir projede white box testine yönelmek daha doğru olurdu.

Soru 4 (derste sözlü, 01:26:09)

Soru: 9. adımdan sonra `Movie`'de kalan fiyat kodu switch-case'i nasıl kaldırılabilir ve bunun bedeli nedir?

Cevap (derste verildi): Switch'i çözmenin yolu bir kalıtım ilişkisi kurmaktır. (a) Replace Type Code with Subclasses: soyut `Movie` ve `MovieRegular`, `MovieNewRelease`, `MovieChildrens`; switch tümden kalkar, ama `Movie(String, int)` kurucusu ve `getPriceCode()` kaybolduğu için testler ve dış arayüz değişir; veri tabanı da etkilenirse geriye doğru uyumluluk kaybolur ve bir ana sürüm kararı gerekir. (b) Replace Type Code with State/Strategy: `Movie` tip kodunu korur, soyut `Price` ve alt sınıfları hesapları üstlenir; `Price` nesnesini seçmek için yalnız basit bir switch kalır, birim testleri değişmez.

Dersin Toparlaması

Ders, kod kusurları listesinin sonuncusu olan Feature Envy ile başladı: bir metodun kendi sınıfından çok başka sınıfların üyelerine erişmesi, sorumlulukların yanlış atandığının belirtisidir ve Move Method ya da Extract Method ile giderilir. Ardından Fowler'ın video kiralama örneği on adımda yeniden yapılandırıldı: switch-case `amountFor` metoduna çıkarıldı, adları düzeltildi, Feature Envy nedeniyle `Rental.getCharge` olarak taşındı, geçici değişkenler sorgularla değiştirildi, bonus hesabı da aynı yolu izledi, toplamlar `getTotalCharge` ve `getTotalBonus` sorgularına çıkarıldı, fiyat ve bonus mantığı switch'in anahtarına sahip olan `Movie`'ye taşındı ve `Customer` ile `Movie` arasındaki doğrudan bağlaşım `Rental.getMovieTitle` ile dolaylı hale getirildi. Her adım küçük tutuldu ve birim testleri değiştirilmeden yeniden çalıştırıldı.

Son adımda tip kodunu kaldırmanın iki yolu karşılaştırıldı: alt sınıflar switch'i tümden kaldırır ama arayüzü ve testleri bozar; Strategy/State çözümü tip kodunu koruyarak testleri bozmadan hesapları dağıtır. Hangisinin seçileceği yalnız teknik değil, geriye doğru uyumluluk ve sürüm politikasıyla ilgili bir karardır. Dersin son bölümünde birinci ara sınavın iki tasarım kalıbı sorusu tartışıldı: ülkeye göre değişen adres, tarih ve telefon sınıfları için Abstract Factory başta olmak üzere birçok kalıbın, hiyerarşik departman yapısı için ise Composite'in (ve Visitor'ın) uygun olduğu; Decorator, Prototype, Adapter, MVC, Iterator ve Chain of Responsibility'nin neden yalnız kısmen uyduğu, Flyweight'in neden uymadığı gerekçeleriyle anlatıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.